EE5203 L07 Worksheet

Timers and non-blocking time - Basri Erdogan

Expected time: 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions.

Assume the Nucleo-F401RE default clock throughout: the TIM2 timer clock is **16 MHz**.

Part A - The pipeline

1. List the five stages of the timer pipeline in order, from the timer clock to the update event, naming the register at each stage.
2. In one sentence each, state what `PSC`, `ARR` and `CNT` do.
3. L06's interrupt path and L07's differ in exactly one box. Name the box that changes, and name three stages that are identical.

Part B - The two divisions

4. Write the formula for the update frequency in terms of the timer clock, `PSC` and `ARR`.
5. Complete the table. Show the arithmetic, not just the answer.

Requirement	Counting frequency	`PSC`	`ARR`
1 ms update	1 MHz		
10 ms update	1 kHz		
25 ms update	10 kHz		
2 s update	1 kHz		

6. A student writes `PSC = 16`, `ARR = 1000` for a 1 ms tick and measures a period slightly longer than 1 ms. Explain the error and compute the exact period their values produce, to four decimal places.
7. Give **two** different `PSC`/`ARR` pairs that both produce a 500 ms update event. State what is traded between them and when the finer one is worth choosing.
8. `ARR` is a 16-bit field on TIM3. What is the longest update period you can obtain on TIM3 from a 16 MHz clock, and with which `PSC`?

Part C - Consuming the event

9. Write the two operations a polling loop must perform on `UIF`, in order, as C statements on `TIM2->SR`.
10. In interrupt mode, who clears `UIF` — your callback, or something else? Name the function.
11. Here is a student's callback. Give three separate problems.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    HAL_Delay(5);
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    ms_counter++;
}
```

12. `ms_counter` above is declared `uint32_t ms_counter;` in `main.c`. The LED schedule works at `-O0` and stops working at `-O2`. Name the missing qualifier and explain, in terms of what the compiler is allowed to assume, why the optimization level changes the behavior.
13. The project uses TIM2 for a 1 ms tick and TIM3 for a 50 ms event. Both fire `HAL_TIM_PeriodElapsedCallback`. Write the first line of the callback body that makes the two distinguishable.

Part D - Scheduling and drift

14. Using a 1 ms `g_ms` tick, write the `while (1)` condition that toggles an LED every 250 ms. Use the wrap-safe form.
15. `g_ms` is a 32-bit millisecond counter. After how long does it wrap? Show the calculation, and explain why the wrap-safe subtraction survives it.

16. A scheduler toggles every 500 ms and reschedules with `t_led = g_ms;`. One loop pass takes 40 ms because of other work.
    a. By how much can a single toggle be late, at worst?
    b. Over the next ten toggles, does the error stay bounded or accumulate?
    c. What changes if the line becomes `t_led += 500U;`?
    d. Give one situation where `t_led = g_ms` is the *better* choice.
17. Two events must run from the same 1 ms tick: one every 100 ms, one every 700 ms. Write the loop body, and state how many `if` statements you need and how many timers.

Part E - Bare metal and evidence

18. Explain each of the five lines, one clause per line:

```
RCC->APB1ENR |= (1U << 0);
TIM2->PSC = 15;
TIM2->ARR = 0xFFFFFFFFU;
TIM2->EGR = (1U << 0);
TIM2->CR1 |= (1U << 0);
```

19. A student omits the `EGR` write. Describe precisely what goes wrong, and how long the wrong behavior lasts.
20. A student writes `RCC->AHB1ENR |= (1U << 0);` instead of line 1. What do they observe in the SFR view, and why?
21. `delay_us()` below is used with `us = 3000000`. The counter runs at 1 MHz and TIM2 is 32-bit. Does it work? Justify with the wrap arithmetic.

```
static inline void delay_us(uint32_t us)
{
    uint32_t start = TIM2->CNT;
    while ((TIM2->CNT - start) < us) { }
}
```

22. Fill in the evidence you would show for each claim.

Claim	SFR observation
The prescaler and period are as configured	
The counter is actually running	
The update interrupt is armed	
The timer's clock is enabled	

Exit self-assessment

- ☐ I can derive `PSC`/`ARR` for any period and check it with the formula.
- ☐ I can explain the +1 rule without looking it up.
- ☐ I can state who clears `UIF` in each of the two modes.
- ☐ I can write a wrap-safe scheduler and explain the drift trade-off.
- ☐ I can justify every line of the bare-metal setup.